

# Transformer-Based CI Build Failure Prediction on TravisTorrent: A Fair Comparison with Parrot, BuildFast, and DL-CIBuild

Wilson Neira

Northeastern University  
Sponsored by Dr. Saad

## Background

Continuous Integration (CI) systems automatically build and test code after changes, but builds can take a long time and failed builds slow developers down. My project studies whether a Transformer-based sequential model can improve CI build failure prediction compared with strong baselines on TravisTorrent-style data.

- TravisTorrent gives a public CI dataset with build outcomes, durations, churn, test signals, and project metadata.
- BuildFast is a strong history-aware tabular baseline with cost-benefit evaluation.
- DL-CIBuild treats build prediction as a sequence problem using LSTM plus GA-based tuning.
- RavenBuild showed that Parrot, which copies the previous build outcome, is a very strong baseline.

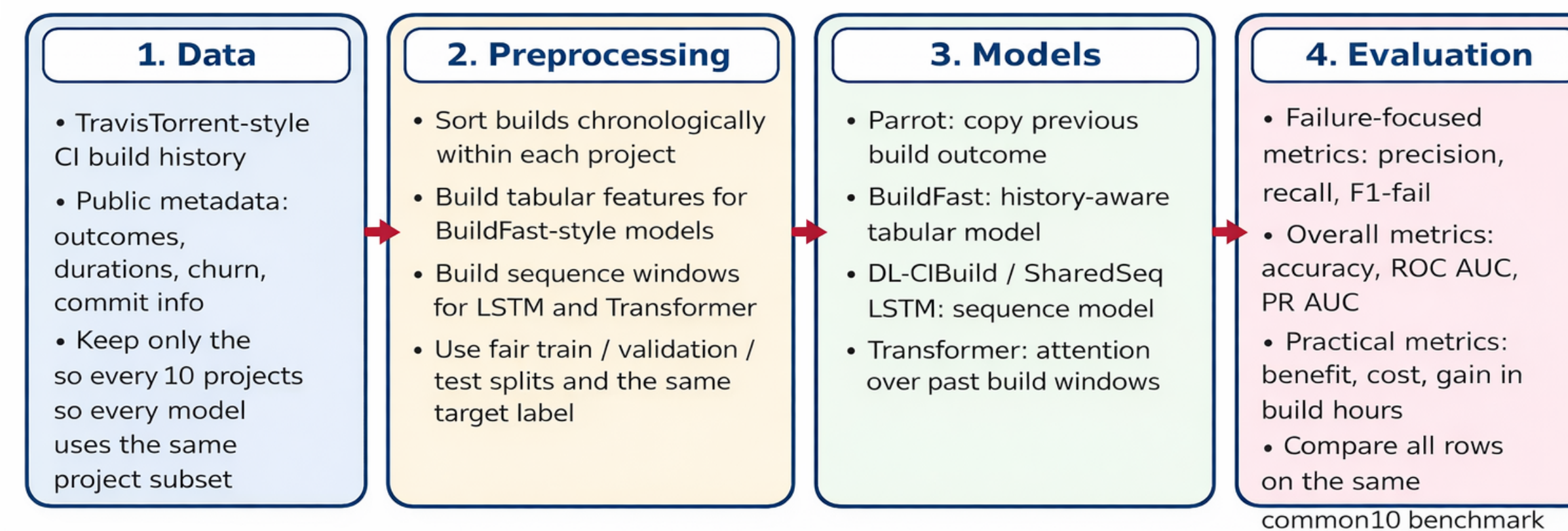
## Methods

I built a fair common10 benchmark so the models are compared on the same 10 projects instead of accidentally using different project subsets.

- Load and clean TravisTorrent-style CI build history.
- Sort builds chronologically within each project to avoid leakage.
- Create history-aware tabular features such as previous outcome, failure streaks, recent failure rates, and duration drift.
- Build shared sequence windows of the last 15 builds for LSTM and Transformer models.
- Evaluate all models with the same failure-focused and cost-aware metrics.

## Methods Overview

Fair common10 benchmark: same 10 projects, same evaluation metrics, different model families



### Why this is fair

All models are evaluated on the same common10 project subset. The main difference is the model family: Parrot uses a simple prior, BuildFast uses history-aware tabular features, and LSTM / Transformer use sequence windows.

- Parrot copies the previous build outcome.
- BuildFast uses history-aware tabular modeling.
- DL-CIBuild uses LSTM with GA-based tuning.
- My Transformer uses sequence windows, attention, and a learned correction on top of a Parrot-style prior.

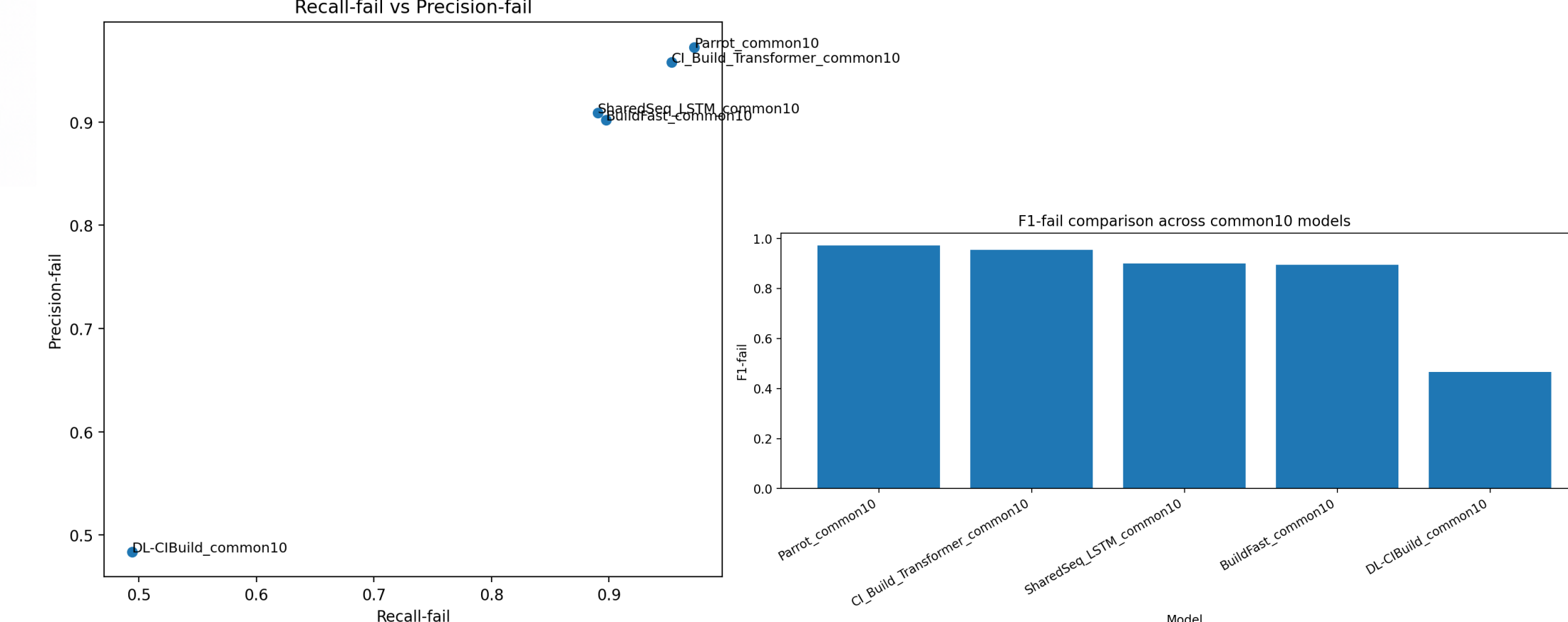
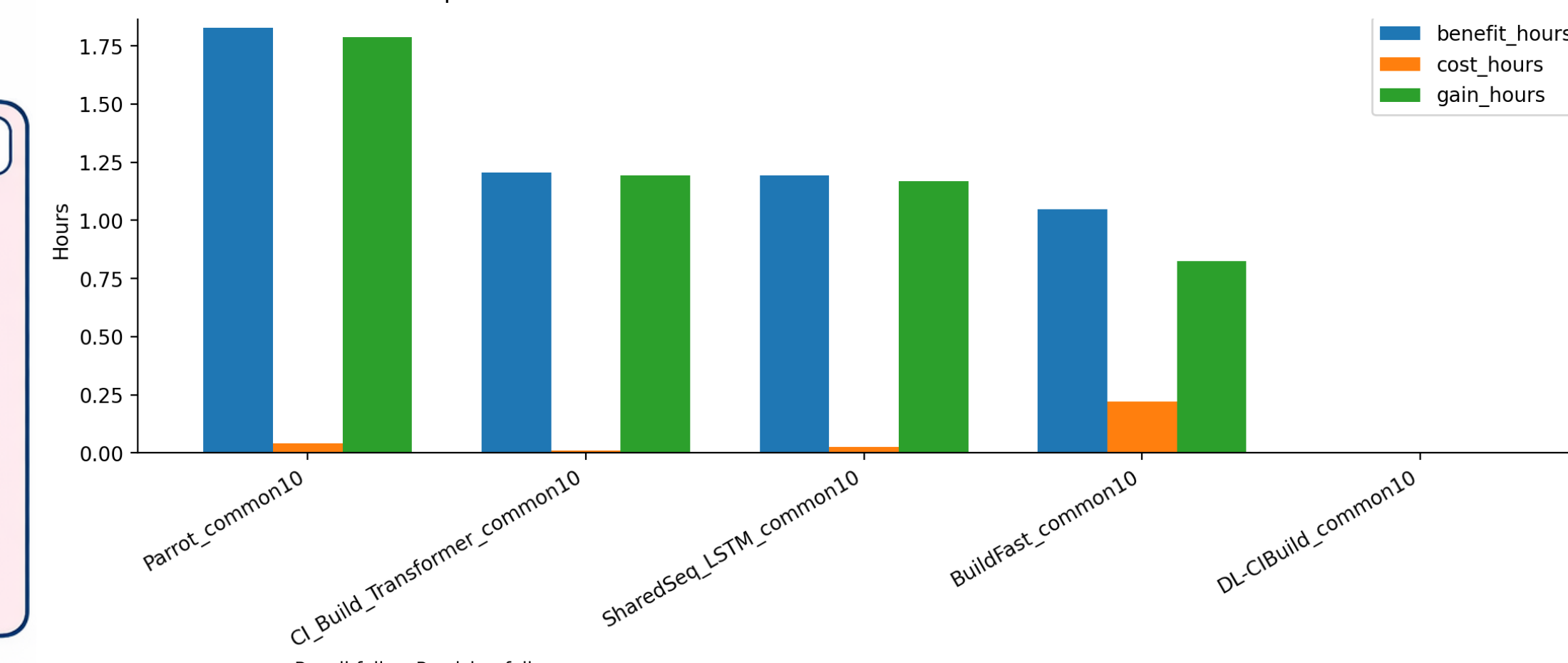
### Future work:

- Improve the Transformer on flip cases, where Parrot is less useful.
- Run more ablations on sequence inputs, depth, heads, and threshold tuning.

## Results

The final result is that Parrot is still the strongest overall row on the fair common10 benchmark, but my Transformer is the strongest learned model.

- Parrot\_common10 is the strongest model overall with F1-fail = 0.972487.
- CI\_Build\_Transformer\_common10 is the best learned model with F1-fail = 0.955377.
- SharedSeq\_LSTM\_common10 = 0.899628, BuildFast\_common10 = 0.894571, and DL-CIBuild\_common10 = 0.467168.
- All main rows use the same common10 benchmark with projects\_processed = 10.
- The Transformer did not beat Parrot, but it did outperform the other learned and adapted baseline rows in this final benchmark.



## References

- [1] Beller, Gousios, Zaidman. *TravisTorrent*. MSR 2017.
- [2] Chen et al. *BuildFast*. ASE 2020.
- [3] Saidani, Ouni, Mkaouer. *DL-CIBuild*. ASE 2022.
- [4] Sun, Habchi, McIntosh. *RavenBuild*. FSE 2024.

## Problem / Research Question

My main question is: can a Transformer-based sequential model improve CI build failure prediction on TravisTorrent compared with Parrot, BuildFast-style, and DL-CIBuild-style baselines?

- Focus on failure-class F1 and recall, not just plain accuracy.
- Keep the comparison fair by using the same common10 project subset.
- Check practical value using benefit, cost, and gain in build hours.
- Be honest about whether the Transformer really beats the strongest baseline.